

# Lab 4 — Introduction to Rust

Language Basics · File Parsing · SSH Log Analyzer Challenge

## Table of contents

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                   | <b>2</b>  |
| <b>2</b> | <b>Setting Up Rust</b>                                                | <b>3</b>  |
| 2.1      | Installation . . . . .                                                | 3         |
| 2.2      | Your First Rust Program . . . . .                                     | 4         |
| <b>3</b> | <b>Rust Language Basics</b>                                           | <b>5</b>  |
| 3.1      | Imports ( <code>use</code> ) . . . . .                                | 5         |
| 3.2      | Variables . . . . .                                                   | 5         |
| 3.3      | Functions . . . . .                                                   | 6         |
| 3.4      | Error Handling . . . . .                                              | 6         |
| 3.5      | <code>Option&lt;T&gt;</code> — Values That Might Be Missing . . . . . | 7         |
| 3.6      | Loops . . . . .                                                       | 7         |
| 3.7      | Conditionals . . . . .                                                | 8         |
| 3.8      | Command-Line Arguments . . . . .                                      | 8         |
| 3.9      | String Formatting . . . . .                                           | 9         |
| <b>4</b> | <b>Working with Files and Text in Rust</b>                            | <b>9</b>  |
| 4.1      | Strings vs. String Slices ( <code>&amp;str</code> ) . . . . .         | 9         |
| 4.2      | Reading a File Line by Line . . . . .                                 | 10        |
| 4.3      | Parsing a Line . . . . .                                              | 11        |
| 4.4      | Counting with a <code>HashMap</code> . . . . .                        | 11        |
| 4.5      | Threads — Processing Multiple Files Concurrently . . . . .            | 12        |
| <b>5</b> | <b>The Challenge — Build an SSH Brute-Force Log Analyzer</b>          | <b>13</b> |
| 5.1      | Overview . . . . .                                                    | 13        |
| 5.2      | Milestone 1 — Parse a Single Line . . . . .                           | 14        |
| 5.3      | Milestone 2 — Parse a Whole File Sequentially . . . . .               | 14        |
| 5.4      | Milestone 3 — Accept a File Path and a Threshold . . . . .            | 15        |

|          |                                                                            |           |
|----------|----------------------------------------------------------------------------|-----------|
| 5.5      | Milestone 4 — Make It Concurrent . . . . .                                 | 15        |
| 5.6      | Milestone 5 — Borrow Instead of Own ( <i>Stretch Goal</i> ) . . . . .      | 15        |
| 5.7      | Milestone 6 — Swap Logs with a Classmate ( <i>Stretch Goal</i> ) . . . . . | 16        |
| <b>6</b> | <b>Putting It All Together — Reference Implementation Outline</b>          | <b>16</b> |
| <b>7</b> | <b>Summary</b>                                                             | <b>18</b> |

# 1 Introduction

**Rust** is a compiled, statically-typed systems programming language first released by Mozilla in 2010, designed around one central promise: memory safety without a garbage collector. It achieves this through its **ownership** and **borrowing** system, enforced entirely at compile time — a whole category of bugs (use-after-free, data races, null pointer dereferences) that plague languages like C and C++ simply will not compile in Rust. Like Go, Rust programs compile to a single self-contained binary with no runtime dependencies.

Rust has become increasingly prominent in security tooling precisely because of these guarantees — memory-safety vulnerabilities have historically been one of the largest sources of real-world CVEs, and Rust eliminates most of that category by construction.

| Tool                 | What It Does                                                                           |
|----------------------|----------------------------------------------------------------------------------------|
| <b>RustScan</b>      | An extremely fast port scanner, built for the same reasons this lab uses Rust at all   |
| <b>ripgrep</b>       | Fast recursive text search, widely used in recon and log analysis                      |
| <b>Sniffglue</b>     | A network packet sniffer built with Rust’s safety guarantees in mind                   |
| <b>cargo-audit</b>   | Scans a Rust project’s dependencies for known vulnerabilities                          |
| Linux kernel drivers | The kernel has begun accepting Rust code specifically for its memory-safety guarantees |

By the end of this lab you will have written your own SSH brute-force log analyzer in Rust — a simplified version of what tools like **fail2ban** and **CrowdSec** do in production, regardless of what language they happen to be written in: read logs, detect abuse, and flag the source.

## **i** Why Rust for Security Tools?

Three properties make Rust a natural fit for network security tools:

- **Safety without a garbage collector:** Rust catches memory-safety and data-race bugs at *compile time*, not runtime — there’s no garbage collector pausing your program, and no crashing in production from a bug a compiler could have caught.
- **Speed:** Rust compiles to native machine code with performance comparable to C and C++, and generally on par with or faster than Go for CPU-bound work.
- **Fearless concurrency:** The same ownership rules that prevent memory bugs also prevent data races between threads — the compiler literally refuses to compile code that shares mutable data unsafely between threads.

### 💡 Coming From the Go Lab?

This lab mirrors the structure of the Go lab — same general shape, same idea of building a small security tool from scratch — but the challenge itself is different by design: instead of a network port scanner, you’ll build a log analyzer, which gives Rust’s ownership and borrowing system real work to do in a way that a simple socket-connect loop doesn’t. You do not need to have done the Go lab first, but if you have, watch for the callouts comparing the two directly.

---

## 2 Setting Up Rust

### 2.1 Installation

Unlike many languages, Rust is **not** installed via **apt**. The Rust team maintains its own toolchain installer and manager, called **rustup**, and actively recommends using it instead of distro packages — distro versions lag badly and don’t give you **cargo** (Rust’s build tool and package manager) in a consistent state.

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

### 📖 A Quick Security Aside

You’re about to pipe a remote script straight into a shell — worth pausing on in a cybersecurity course! This is the Rust project’s own official, documented install method (served over TLS from **rustup.rs**, which redirects to the Rust Foundation’s infrastructure), and it’s about as legitimate as this pattern gets. But it’s a good habit to know *why* you trust a command before running it as yourself, rather than just because a tutorial told you to. If you’d rather inspect it first, you can download the script and read it before running it:

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs -o install-rust.sh
less install-rust.sh
sh install-rust.sh
```

Follow the prompts (the default install option is fine), then restart your terminal (or run `source "$HOME/.cargo/env"`) so the `rustc` and `cargo` commands are on your PATH.

Verify the installation succeeded:

```
rustc --version
cargo --version
```

You should see output for both, with matching version numbers.

## 2.2 Your First Rust Program

Create a new file called `hello.rs`:

```
nano hello.rs
```

Type the following:

```
fn main() {
    println!("Hello, World!");
}
```

Compile it, then run the resulting binary:

```
rustc hello.rs
./hello
```

### **i** rustc vs. cargo

For this lab's small, single-file programs, calling `rustc` directly (like Go's `go build`) is simple and works fine. For any real project, Rust developers use **Cargo**, Rust's build tool and package manager, instead:

```
cargo new hello_cargo    # scaffolds a new project with a Cargo.toml
cd hello_cargo
cargo run                # compiles and runs in one step
cargo build --release    # compiles an optimized binary
```

Cargo manages dependencies, build profiles, and project structure automatically. You'll see `cargo` referenced constantly outside this lab — it's worth recognizing even though we won't need it for these exercises (everything we use lives in Rust's standard library, so there are no external dependencies to manage).

---

## 3 Rust Language Basics

If you already know Go, C, Python, or Java, a lot of Rust's syntax will feel familiar. This section highlights the parts that are different or worth paying attention to before the challenge.

### 3.1 Imports (`use`)

Rust brings items from the standard library or other crates into scope with `use`.

```
use std::fs::File;           // opening files
use std::env;                // OS interaction (args, exit)
use std::collections::HashMap; // key-value storage
```

#### Unused Imports: Warning, Not Error

Unlike Go, an unused `use` statement in Rust produces a **warning**, not a compile error — your code will still build. It's good practice to clean these up, but it won't block you the way it does in Go.

### 3.2 Variables

Rust variables are **immutable by default** — a deliberate safety choice. Add `mut` to make one mutable.

```
// Immutable by default
let host = "localhost";
let open = true;

// Explicit type annotation
let port: u16 = 80;

// Mutable - required if you intend to reassign or modify later
let mut min_port = 1;
min_port += 1;
```

### Immutability by Default

This is one of Rust's core safety features: if a variable isn't marked `mut`, the compiler guarantees it never changes after being set. This eliminates an entire class of bugs where a value is modified somewhere you didn't expect.

## 3.3 Functions

```
fn greet(name: &str, times: u32) -> String {
    let mut result = String::new();
    for _ in 0..times {
        result.push_str(&format!("Hello, {}\\n", name));
    }
    result
}
```

## 3.4 Error Handling

Rust, like Go, has no exceptions. Instead of Go's (value, error) return pattern, Rust uses the `Result<T, E>` type — a single return value that is *either* a success (`Ok`) *or* a failure (`Err`).

```
fn divide(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        return Err("cannot divide by zero".to_string());
    }
    Ok(a / b)
}
```

The idiomatic way to handle a `Result` is `match`:

```
match divide(10.0, 0.0) {
    Ok(result) => println!("Result: {}", result),
    Err(e) => println!("Error: {}", e),
}
```

You will see this pattern constantly in Rust — especially around parsing and file I/O, where many things can go wrong (just like in Go).

### 3.5 `Option<T>` — Values That Might Be Missing

Rust has a second type in the same family as `Result`: `Option<T>`, for a value that might simply not be there — no error involved, just “yes” (`Some(value)`) or “no” (`None`). Anything that searches for something uses it:

```
let numbers = vec![1, 2, 3];
let found: Option<i32> = numbers.iter().find(|&n| n == 5);

match found {
    Some(n) => println!("Found {}", n),
    None => println!("Not found"),
}
```

You’ll use `Option` constantly once you start parsing text later in this lab — `.find()`, splitting strings, and looking things up all return `Option` rather than crashing or returning a sentinel value like `-1`.

### 3.6 Loops

Rust has `loop`, `while`, and `for`. Unlike Go, these are three separate keywords rather than one `for` covering everything.

```
// Range-based loop (most common - like Python's `for i in range(...)`)
for i in 0..10 {
    println!("{}", i);
}

// While-style loop
let mut count = 0;
```

```
while count < 5 {
    count += 1;
}

// Infinite loop (useful for servers) - `loop` is its own keyword
loop {
    // runs forever until a `break`
}
```

### 3.7 Conditionals

```
let port = 22;

if port == 22 {
    println!("SSH");
} else if port == 80 {
    println!("HTTP");
} else {
    println!("Unknown");
}
```

Rust's `match` is the equivalent of Go's `switch`, and is used constantly:

```
match port {
    21 => println!("FTP"),
    22 => println!("SSH"),
    80 => println!("HTTP"),
    3306 => println!("MySQL"),
    _ => println!("Unknown service"),
}
```

### 3.8 Command-Line Arguments

`std::env::args()` returns an iterator over the command-line arguments. Like Go's `os.Args`, the first element is the program name itself.

```
use std::env;

fn main() {
```

```

let args: Vec<String> = env::args().collect();
if args.len() < 2 {
    println!("Usage: ./program <name>");
    std::process::exit(1);
}
let name = &args[1];
println!("Hello, {}", name);
}

```

```

rustc hello.rs && ./hello Alice
# Output: Hello, Alice

```

### 3.9 String Formatting

The `format!` macro builds a formatted string (like Go's `fmt.Sprintf` or Python's f-strings). `println!` uses the same placeholder syntax.

```

let host = "192.168.1.1";
let port = 80;
let address = format!("{}", host, port);
// address is now "192.168.1.1:80"

```

Common placeholders:

| Placeholder        | Meaning                                                             |
|--------------------|---------------------------------------------------------------------|
| <code>{}</code>    | Display formatting (most common — works for numbers, strings, etc.) |
| <code>{:?}</code>  | Debug formatting (useful for printing structs, vectors, tuples)     |
| <code>{:x}</code>  | Hexadecimal                                                         |
| <code>{:.2}</code> | Float, rounded to 2 decimal places                                  |

## 4 Working with Files and Text in Rust

### 4.1 Strings vs. String Slices (&str)

Before parsing anything, it's worth being explicit about a distinction Go doesn't have: Rust has two string types.

```
// `String` is an owned, growable, heap-allocated string.
let owned: String = String::from("Failed password for root");

// `&str` is a borrowed *view* into string data - either a string
// literal (baked into the binary) or a slice of an existing String.
let literal: &str = "Failed password for root";
let slice: &str = &owned[0..14];
```

Rust is strict about this because it's central to the ownership system: a `&str` never owns the data it points to, so the compiler must always be able to prove the data it borrows from is still alive. You'll see both types constantly — function parameters are usually `&str` (so callers can pass either kind in), while anything you build up or need to store for later is usually `String`.

## 4.2 Reading a File Line by Line

```
use std::fs::File;
use std::io::{BufRead, BufReader};

fn main() {
    let file = File::open("/var/log/auth.log").expect("could not open log file");
    let reader = BufReader::new(file);

    for line in reader.lines() {
        let line = line.expect("could not read line");
        println!("{}", line);
    }
}
```

`BufReader` wraps the file so lines are read efficiently rather than one syscall at a time. `.lines()` gives you an iterator of `io::Result<String>` — each line comes back as an *owned* `String`, since the reader's internal buffer gets reused for the next line and can't be safely borrowed from.

### 💡 Generate Some Test Data First

A fresh VM's `auth.log` probably has few or no failed logins yet. Before testing your analyzer, generate some: try `ssh fake_user@localhost` a few times with a wrong password (it will fail, which is exactly what you want) so your log has data to detect.

### **i** No /var/log/auth.log on Your System?

Some systems route SSH logs through the systemd journal instead of a flat log file. If /var/log/auth.log doesn't exist, export the journal to a text file and use that path instead:

```
sudo journalctl -u ssh --no-pager > ~/auth.log
```

## 4.3 Parsing a Line

A failed SSH login looks like this in the log:

```
Jan 15 10:23:45 myhost sshd[12345]: Failed password for invalid user admin from 203.0.113.42
```

We want the IP address after `from`. `.contains()`, `.split()`, and `Option` are the tools for this:

```
fn extract_ip(line: &str) -> Option<&str> {
    if !line.contains("Failed password") {
        return None;    // not a failed-login line - skip it
    }

    let after_from = line.split("from ").nth(1)?;    // everything after "from "
    let ip = after_from.split(' ').next()?;          // the first word of that
    Some(ip)
}
```

The `?` here works the same way it does with `Result`: if `.split("from ").nth(1)` comes back `None` (the line didn't contain "from"), the function returns `None` immediately instead of panicking.

Notice `extract_ip` returns `Option<&str>` — a *slice* borrowed from the original `line`, not a new `String`. No copying happens; we're just pointing at part of the data that's already there.

## 4.4 Counting with a HashMap

```

use std::collections::HashMap;

let mut counts: HashMap<String, u32> = HashMap::new();

for line in reader.lines() {
    let line = line.expect("could not read line");
    if let Some(ip) = extract_ip(&line) {
        *counts.entry(ip.to_string()).or_insert(0) += 1;
    }
}

```

#### 💡 The `.entry().or_insert()` Pattern

`counts.entry(ip.to_string())` looks up `ip` in the map. `.or_insert(0)` inserts a 0 if it isn't already there, and either way returns a mutable reference to the value — which is why `*counts.entry(...).or_insert(0) += 1` works: dereference with `*` and increment in place. This is the idiomatic way to build a frequency count in Rust.

Note the `ip.to_string()` call: `extract_ip` gave us a borrowed `&str` pointing into `line`, but `line` itself doesn't live past this loop iteration. To store the IP in the `HashMap` for later, we need our own owned copy — hence `.to_string()`. This is the ownership system in action: the compiler won't let you store a borrowed reference for longer than the data it points to is guaranteed to exist.

## 4.5 Threads — Processing Multiple Files Concurrently

If you have several log files to analyze (your own VM plus a few classmates', say), processing them one at a time is wasteful when they're completely independent of each other. Spawn one thread per file, and have each thread return its own results:

```

use std::collections::HashMap;
use std::thread;

fn analyze_file(path: String) -> HashMap<String, u32> {
    // ... open path, read lines, build and return a HashMap ...
    HashMap::new()
}

fn main() {
    let paths = vec!["auth1.log".to_string(), "auth2.log".to_string()];
}

```

```

let mut handles = vec![];
for path in paths {
    let handle = thread::spawn(move || analyze_file(path));
    handles.push(handle);
}

let mut combined: HashMap<String, u32> = HashMap::new();
for handle in handles {
    let file_counts = handle.join().unwrap();
    for (ip, count) in file_counts {
        *combined.entry(ip).or_insert(0) += count;
    }
}

// `combined` now holds totals across every file
}

```

This uses the same closure-ownership rule you’d hit anywhere in Rust: the closure passed to `thread::spawn` must own everything it uses, so `path` is moved in with `move` rather than borrowed — the same requirement as capturing a loop variable, just applied to a real value instead of a toy example.

### ! Notice What We *Didn't* Need

Each thread owns its own `path` and returns its own, independently-owned `HashMap` — nothing is shared *while* the threads are running. We only combine results afterward, in the main thread, once each `JoinHandle` hands its data back. Because of this, we never needed `Arc`, and never needed a lock (`Mutex`) to protect shared, mutable data — Rust’s ownership model naturally pushed us toward a “give each thread its own data, merge the results afterward” design, which sidesteps locking entirely. This is a very common and very idiomatic Rust concurrency pattern.

## 5 The Challenge — Build an SSH Brute-Force Log Analyzer

### 5.1 Overview

Working individually or in groups of up to five, your task is to build a tool in Rust that scans SSH authentication logs for signs of a brute-force attack — the same kind of detection real intrusion-detection tools perform. Your analyzer should:

1. Accept one or more **log file paths** as command-line arguments.
2. Parse each file for failed SSH login attempts and the IP address responsible.
3. Count failed attempts **per IP address**.
4. Flag any IP at or above a threshold (e.g. 5 failed attempts) as a likely brute-force source.
5. Process multiple files **concurrently** using threads.

## 5.2 Milestone 1 — Parse a Single Line

Before worrying about whole files, write a function that takes one hardcoded log line and returns the offending IP address if it's a failed-login line (or `None` otherwise).

You will need:

- `.contains()` to check whether the line is a failed-login line at all
- `.split(...)` and `.nth(...)` (or similar) to isolate the IP address
- `Option<&str>` as your return type

**Expected output**, given the sample line above:

```
Some("203.0.113.42")
```

## 5.3 Milestone 2 — Parse a Whole File Sequentially

Extend your program to open a real log file, read it line by line, and build a `HashMap<String, u32>` counting failed attempts per IP. Print the map when you're done.

You will need:

- `File::open` and `BufReader`
- `.lines()` to iterate over the file
- The `.entry().or_insert()` pattern to count

### Test Against Lab 3

You already set up OpenSSH in Lab 3. Generate a few failed logins locally (see the “Generate Some Test Data First” callout above) and confirm your program counts them correctly before moving on.

## 5.4 Milestone 3 — Accept a File Path and a Threshold

Hardcoded paths are fine for testing but useless as a real tool. Update your program to accept the log file path from the command line, and print only the IPs that meet or exceed a threshold (e.g. 5 attempts) as likely attackers.

```
rustc analyzer.rs && ./analyzer /var/log/auth.log
```

**Expected output** (formatting is up to you):

Possible brute-force sources:

203.0.113.42 - 7 failed attempts

198.51.100.9 - 5 failed attempts

## 5.5 Milestone 4 — Make It Concurrent

Update your program to accept **multiple** file paths (`env::args().skip(1)` gets you all of them at once), spawn one thread per file using the pattern from the “Threads — Processing Multiple Files Concurrently” section above, and merge the results into one combined report.

```
./analyzer auth1.log auth2.log auth3.log
```

### Where Do the Extra Log Files Come From?

Copy your `auth.log` a couple of times under different names to test with multiple “files” before trying it with a classmate’s real log.

Time your analyzer against a handful of files to confirm the threads are actually running in parallel rather than one after another:

```
time ./analyzer auth1.log auth2.log auth3.log
```

## 5.6 Milestone 5 — Borrow Instead of Own (*Stretch Goal*)

Everything so far immediately converts each match into an owned `String` for storage. As a stretch, try representing each failed attempt as a struct that *borrow*s directly from the line it came from, instead of copying:

```
struct LoginAttempt<'a> {
    ip: &'a str,
    raw_line: &'a str,
}
```

The `<'a>` is a **lifetime parameter** — it tells the compiler “a `LoginAttempt` cannot outlive the string data it borrows from.” Try building a `Vec<LoginAttempt>` for one file’s contents and see what the compiler says if you try to use that `Vec` after the `String` holding the file’s contents has gone out of scope. This is Rust’s ownership system at its most visible — and exactly the kind of compiler pushback that doesn’t have an equivalent in Go.

## 5.7 Milestone 6 — Swap Logs with a Classmate (*Stretch Goal*)

Trade log files with a classmate (or use logs from a shared Cyber Storm environment, if available) and run your analyzer against theirs.

### ! A Note on Log Privacy

Log files can contain usernames, IP addresses, and other information about real systems and real people. Only analyze logs from your own VM, a classmate’s VM in this lab environment, or a source you have explicit permission to examine — the same standard as the rest of this course’s network activity, consistent with Louisiana Tech’s acceptable use policy.

## 6 Putting It All Together — Reference Implementation Outline

The structure below is a suggested skeleton. Fill in the blanks as you work through the milestones. Do not copy-paste — write it yourself and understand each line.

```
use std::collections::HashMap;
use std::env;
use std::fs::File;
use std::io::{BufRead, BufReader};
use std::thread;

const THRESHOLD: u32 = 5;
```

```

// extract_ip returns the source IP of a failed-login line, or None
// if the line isn't a failed-login line.
fn extract_ip(line: &str) -> Option<&str> {
    if !line.contains("Failed password") {
        return None;
    }
    let after_from = line.split("from ").nth(1)?;
    after_from.split(' ').next()
}

// analyze_file reads one log file and returns a count of failed
// attempts per IP address.
fn analyze_file(path: String) -> HashMap<String, u32> {
    let mut counts = HashMap::new();

    let file = match File::open(&path) {
        Ok(f) => f,
        Err(_) => return counts,
    };
    let reader = BufReader::new(file);

    for line in reader.lines() {
        let line = match line {
            Ok(l) => l,
            Err(_) => continue,
        };
        if let Some(ip) = extract_ip(&line) {
            *counts.entry(ip.to_string()).or_insert(0) += 1;
        }
    }

    counts
}

fn main() {
    // TODO Milestone 3/4: Read one or more file paths from env::args()
    // TODO Milestone 4: Spawn a thread per file with analyze_file, collect JoinHandles
    // TODO Milestone 4: Merge each thread's HashMap into one combined HashMap
    // TODO Milestone 3: Filter and print IPs at or above THRESHOLD
}

```

## 7 Summary

| Concept                    | Rust Feature Used                                                               |
|----------------------------|---------------------------------------------------------------------------------|
| Reading a file             | <code>File::open</code> + <code>BufReader</code>                                |
| Borrowed vs. owned strings | <code>&amp;str</code> vs. <code>String</code>                                   |
| Parsing a line             | <code>.split()</code> , <code>.contains()</code> , <code>Option&lt;T&gt;</code> |
| Counting per key           | <code>HashMap</code> + <code>.entry().or_insert()</code>                        |
| Concurrent file processing | <code>thread::spawn</code> returning a value + <code>JoinHandle&lt;T&gt;</code> |
| Command-line input         | <code>env::args()</code>                                                        |
| Error handling             | <code>Result&lt;T, E&gt;</code> + <code>match</code>                            |

### 💡 Going Further

- **Real-time monitoring:** Instead of analyzing a static file, research a crate like `notify` to watch `/var/log/auth.log` for new lines as they're written and flag attacks as they happen.
- **`Arc<Mutex<...>>`:** This lab's threads never share mutable state — they own their input and return their output. Research how you'd write this *differently* with all threads incrementing one shared `HashMap` directly (you'll need `Arc<Mutex<HashMap<...>>>`), and think about why the approach used in this lab avoids that complexity.
- **Auto-block offenders:** Real tools like `fail2ban` don't just detect brute-forcers — they act, typically by adding a firewall rule. Research how you might shell out to `iptables` or `ufw` from Rust to block an IP automatically (and think about the risk of a bug in your detection logic blocking innocent traffic).
- **Compare with Go:** If you also did the Go lab, think about how you'd write this same log analyzer in Go. Would `os.Args`, a `map[string]int`, and goroutines make this easier or harder than it was in Rust?